THE RED BOOK

Functional Programming in Scala, for someone who will never write Scala

Boris Cherny created Claude Code. Asked for the one technical book that shaped him most as an engineer, he named this one, and then said you will probably never use the language. Here is what he actually meant, and why it matters to someone who directs agents instead of writing code.

CONTENTS

01 The vending machine

Picture a vending machine. Same coin, same button, same snack, every single time.

- A **pure function** (a step that only turns input into output) is that machine. Nothing else happens.
- It never reaches into your pocket, never edits yesterday's receipt, never changes the weather outside.
- A **side effect** (anything a step does besides return its answer) is the machine that also quietly double charges your card.
- Pure steps are the ones you can trust without watching them run. Impure steps are the ones you have to babysit.

02 The label

Before you buy, you read the label on the slot. What goes in, what comes out. You do not need to see the wiring.

- A **type signature** (the label: what a step takes and what it gives back) is a promise made before any work is done.
- Boris Cherny, who created Claude Code, puts it directly: when he codes, he thinks in types, and the signature matters more than the code under it.
- Write the label first and most wrong answers stop being possible. A machine labelled coin in, snack out cannot hand you a receipt.
- This is the whole trick. Describe WHAT precisely enough and the HOW mostly forces itself.

15
Chapters, across four parts

2014
First edition, second landed 2023

1
Book Boris names as his biggest influence

03 The factory

One machine is boring. A row of them, where each one feeds the next, is a factory.

- **Composition** (chaining small steps so one output becomes the next input) is how tiny reliable pieces become a large reliable system.
- **Immutability** (never editing a thing already handed out) means the snack in your hand cannot change after the fact. You make a new one instead.
- The book does not lecture on any of this. It hands you exercises and makes you build the pieces yourself, from nothing.
- That is why people say it changes how you think rather than what you know.

04 The catch

Lead with the honest part, because this book is not a weekend read.

- It is **hard**, and the exercises are the entire product. Skim it and you get almost nothing.
- The Scala is beside the point. Boris says plainly you will probably never use Scala day to day.
- The later chapters go abstract fast. Most readers get the value from the first half and stop, and that is a fine outcome.
- For you the payoff is not code. It is the habit of writing the contract before the work, which is exactly what a plan and a rule file already do.

THE ONE IDEA TO KEEP

Write the label before you build the machine. A step that only turns input into output can be trusted without watching it. Everything else has to be supervised.

WHY THIS LANDS FOR YOU

Boris's number one rule for Claude Code is give the agent a way to verify its own work. That only works if the work has a **checkable contract**. This book is 15 chapters of learning to write that contract first. Same idea, different room.

SOURCES & NOTE

Sources: Functional Programming in Scala, Paul Chiusano and Runar Bjarnason, Manning. First edition 2014, second edition 2023 with Michael Pilquist; Boris Cherny, creator of Claude Code, in a recorded interview. Quotes here are relayed from a public transcript, so treat the wording as close paraphrase rather than verbatim.

Educational. The book is a working textbook, not a reference. Its value is in doing the exercises, so read it with a keyboard open or do not start it.

Glossary

Functional programming FP A style built from steps that only turn input into output, with no hidden extras.

Pure function A step that always gives the same answer for the same input and does nothing else.

Side effect Anything a step does besides return its answer, such as writing a file or charging a card.

Type The kind of thing a value is, such as a number, a name, or a list of trades.

Type signature The label on a step: what it takes in and what it gives back.

Composition Chaining small steps so the output of one becomes the input of the next.

Immutability Never editing a value already handed out. You build a new one instead.

Referential transparency You can swap a call for its answer and nothing changes. The formal name for purity.

The Red Book The nickname for this book, after its cover.

Scala The language the book uses. A vehicle for the ideas, not the point of them.